

GridWeaver

Virtual Thread IoT Microgrid State Engine

Final Project Report

Java Development Internship Project Submission

Project Area	Java Development / Concurrency / IoT Simulation
Technology	Java 21, Maven, JUnit 5
Core Feature	Virtual-thread based asynchronous sensor updates
Prepared For	Internship Project Evaluation

1. Executive Summary

GridWeaver is a Java 21 microgrid state engine designed to demonstrate practical Java development workflows, object-oriented design and modern concurrency. The application models solar generators, batteries and electrical loads as IoT devices. Sensor readings are submitted asynchronously through Java virtual threads, while shared device state is protected using thread-safe Java collections and atomic values.

The engine derives a live snapshot containing generation, load, battery contribution, net balance, grid import and the number of online devices. The project also contains a repeatable benchmark comparing a bounded platform-thread pool with virtual threads for a high-concurrency, I/O-like workload.

The implementation is intentionally compact so it can be demonstrated reliably during an internship evaluation while still providing clear extension points for REST, MQTT, database persistence and a web dashboard.

2. Project Objective

The objective is to provide practical exposure to real-world Java development workflows by designing, implementing, testing and documenting a concurrency-oriented IoT application.

Objective	Implementation in GridWeaver
Java application development	Maven project structure, Java 21 source and modular packages
OOP principles	Device model, enum-based device types, immutable snapshot record
Backend functionality	MicrogridStateEngine handles registration, updates and aggregation
Concurrency	Virtual-thread executor plus ConcurrentHashMap and AtomicReference
Testing	JUnit 5 tests and dependency-free smoke tests
Documentation	README, project documentation, final report and presentation

3. Scope of Work

The assigned scope calls for understanding project requirements, designing and developing Java applications, applying OOP and backend functionality, integrating APIs/libraries where required, testing and debugging, maintaining version control and documentation, providing progress evidence, and completing the project within the timeline.

GridWeaver addresses the core Java and concurrency portions of this scope. A database and external API are deliberately treated as optional extensions because the current project objective can be completed with an in-memory state engine.

Functional Scope

- Register solar, battery and load devices.
- Maintain online/offline state and power readings.
- Process sensor updates asynchronously using virtual threads.
- Calculate power balance and grid import.
- Generate a readable health report and state snapshot.
- Run smoke tests and a concurrency benchmark.

Out of Scope for the 2-hour build

- Physical IoT hardware integration.
- Persistent database storage.
- Authentication/authorization.
- Production-grade cloud deployment.

4. System Architecture

The architecture follows a simple event-to-state pipeline. Device objects hold the latest readings. Sensor updates are submitted to an executor backed by virtual threads. The state engine aggregates all online devices into a consistent snapshot.

IoT Devices	→	Virtual Thread Sensor Tasks	→	Thread-safe Device Registry	→	Microgrid State Engine	→	Snapshot / Health Report
Solar / Battery / Load		Async updates		ConcurrentHashMap + AtomicReference		Aggregation + balance		Status + metrics

Core formula

balance = generation + batteryPower – load

If the balance is negative, the absolute deficit is represented as grid import. If the balance is positive, the state is reported as surplus. This makes the project easy to explain in a live demonstration.

Main Components

Component	Responsibility
Device	Represents an IoT endpoint and stores the latest power reading.
DeviceType	Defines SOLAR_PANEL, BATTERY, LOAD and GRID_IMPORT.
MicrogridStateEngine	Registers devices, accepts asynchronous updates and calculates snapshots.
MicrogridSnapshot	Immutable record containing derived state values.
VirtualThreadBenchmark	Compares platform-thread pool and virtual-thread execution.
Main	Runs the demonstration scenario.

5. Technology and Implementation

Java 21: chosen for native virtual-thread support and modern language features such as records and switch expressions.

Maven: provides a conventional Java project layout and dependency management. JUnit 5 is declared for automated tests.

Concurrency: ConcurrentHashMap protects the device registry, AtomicReference protects mutable device readings, and Executors.newVirtualThreadPerTaskExecutor() handles asynchronous telemetry tasks.

Validation: negative power readings are rejected. Unknown device IDs do not crash the update task.

Example execution flow

Step	Action
1	Create the state engine.
2	Register solar, battery and load devices.
3	Submit sensor updates as independent virtual-thread tasks.
4	Wait briefly for asynchronous tasks to apply the readings.
5	Generate a MicrogridSnapshot.
6	Print health/status information and optionally run the benchmark.

6. Testing and Performance Evaluation

The project contains two verification paths. JUnit 5 tests cover grid-import calculation and battery contribution. A zero-dependency smoke-test class is also included so the core logic can be checked with only JDK 21 installed.

Reference verification

Check	Result
Java 21 compilation	PASS
Grid import calculation	PASS
Battery contribution	PASS
Asynchronous sensor update	PASS
Smoke test suite	3/3 PASS
Reference benchmark	Platform pool: 224 ms; Virtual threads: 28 ms

The reference benchmark used 1,000 short 20 ms I/O-like tasks. It measured approximately 224 ms using a fixed 100-thread platform pool and 28 ms using virtual threads in the verification environment. These numbers are hardware/OS/JDK dependent and should be rerun on the final demo machine before presenting them as local measurements.

Why virtual threads matter

The workload contains many tasks that spend most of their lifetime waiting. Virtual threads allow a large number of such tasks to be represented without requiring a matching number of heavyweight platform threads. The benchmark is therefore an illustration of concurrency behaviour, not a universal performance guarantee.

7. Deliverables Mapping

The assignment lists source code, project documentation, project report, presentation, model evaluation results where applicable, and a final project demonstration. The submission package contains each applicable item.

Required deliverable	Included
Source Code	Yes — complete Maven Java 21 source tree
Project Documentation	Yes — docs/Project_Documentation.md
Project Report	Yes — this PDF
Presentation (PPT)	Yes — GridWeaver_Presentation.pptx
Model Evaluation Results	Not applicable as an ML model; concurrency benchmark included instead
Final Project Demonstration	Yes — run.bat and run-benchmark.bat provide a live demo

8. Future Enhancements

The next iteration can add a Spring Boot REST API, MQTT telemetry ingestion, PostgreSQL/TimescaleDB persistence, a React/WebSocket dashboard, battery state-of-charge, device failure simulation, alerting, Docker packaging and CI/CD. These extensions would turn the educational simulator into a more complete production-style IoT platform.

9. Conclusion

GridWeaver provides a compact but demonstrable Java development project aligned with the supplied internship assignment. It covers OOP, backend state management, thread-safe data handling, Java 21 virtual threads, testing, documentation and performance evaluation. The implementation is small enough to understand end-to-end during a final review, while the architecture gives clear directions for future API, database and UI integration.

Final demonstration checklist

- Open the project folder in VS Code/IntelliJ.
- Confirm JDK 21 is installed.
- Run self-test or run.bat.
- Explain the Device → Virtual Thread → State Engine flow.
- Run the benchmark if time permits.
- Show README, documentation, report and presentation as evidence of deliverables.

End of Report